

100 Days of Linux - Day 038

Title: Processing Structured Data with awk

Today's Goal

Learn how to extract, filter, and process structured text using the **awk** command.

Why This Matters

Data engineers use **awk** to analyze CSV files, logs, and reports directly from the terminal without opening spreadsheets.

Command of the Day: awk

Syntax: `awk 'pattern {action}' filename`

Common Examples:

`awk '{print $1}' employees.txt` (print first column)

`awk -F',' '{print $2}' employees.csv` (print second CSV column)

`awk -F',' '$4>50000 {print $2, $4}' employees.csv` (filter by salary)

`awk 'NR==1' employees.csv` (print first line)

Explanation:

\$1, \$2, \$3... represent fields. The **-F** option specifies the field delimiter, such as a comma in CSV files.

Beginner Lesson

Think of **awk** as a tool that reads one line at a time, splits it into columns, and lets you choose which columns to display. It is one of the most powerful commands for working with structured data.

Practice Questions

1. Create `employees.csv` with the columns ID, Name, Department, and Salary. Display only the Name column using **awk**.
2. Display the Name and Salary columns from `employees.csv`.
3. Show only employees whose salary is greater than 50000.
4. Print only the first line (header) of `employees.csv` using **awk**.
5. Display your current working directory after completing today's tasks.

Hints

Remember to use `-F','` for CSV files. Fields start at \$1, not \$0.

Mini Challenge

Create `sales.csv` with Product, Category, Quantity, and Price columns. Use **awk to print Product and Price, then display only products with Price greater than 1000.**

Common Mistakes

Forgetting to specify the correct delimiter or using the wrong field number.

Did You Know?

The awk language was named after its creators: Alfred Aho, Peter Weinberger, and Brian Kernighan.

Pro Tip

Combine awk with grep, sort, and wc to build efficient command-line data processing pipelines.

Answer Key (Instructor Only)

Q1: `awk -F',' '{print $2}' employees.csv`

Q2: `awk -F',' '{print $2, $4}' employees.csv`

Q3: `awk -F',' '$4>50000 {print $2, $4}' employees.csv`

Q4: `awk 'NR==1' employees.csv`

Q5: `pwd`

Homework

Create three CSV files and practice extracting different columns and filtering rows using awk.

Next Day Preview

Tomorrow you'll complete a practical text-processing assignment combining sort, uniq, wc, cut, paste, tr, sed, and awk.